The Mathematics of tagd

A Formal Description of the Mathematical Structures
in the tagd Semantic-Relational System

Isaac N Colley

April 2026

Abstract

The tagd system was built from engineering instincts about how knowledge should be structured. This paper makes the mathematics of those instincts explicit. Every claim here is grounded in the tagd source code and the TAGL language specification. The core thesis is simple: *information is structural*. A tag's meaning is not intrinsic — it is determined entirely by its position in the tagspace, by its relations to other tags. This is not a metaphor. It is a precise mathematical claim, and the sections below give it precise mathematical form, identifying in the source code the structures of set theory, partial orders, and Alexandrov topology.

Contents

1 Primitives **2**
1.1 Tags . . . 2
1.2 The Root and the Axiom . . . 2

2 The Subordinate Relation and the Rooted Tree **3**
2.1 Subordinate Relations . . . 3
2.2 The Tagspace as a Rooted Tree . . . 3

3 Rank: Encoding the Tree in Strings **3**
3.1 The Rank Function . . . 3
3.2 The Prefix Partial Order . . . 4
3.3 Verifying the Partial Order Axioms . . . 4
3.4 Lexicographic Total Order . . . 5

4 The Tagspace Topology **5**
4.1 From Partial Order to Topology . . . 5
4.2 Verifying the Topology Axioms . . . 5
4.3 Specialization Order . . . 5
4.4 Continuity as Meaning-Preservation . . . 6

5 The Hard Tag Base: A Universal Subspace **6**
5.1 Hard Tags . . . 6
5.2 The Hard Tag Subspace . . . 6
5.3 Interoperability via the Common Base . . . 6

6 Predicates: Fibered Structure **7**
6.1 Relators Are Tags . . . 7
6.2 Predicates as Horizontal Relations . . . 7
6.3 The Two-Layer Structure . . . 7

7 Structural Identity **7**
7.1 Identity Is Position . . . 7
7.2 The Canonical Form . . . 8

8 Deeper Structures **8**
8.1 The Adjunction Between Order and Topology . . . 8
8.2 Topological Properties of the Tagspace . . . 9
8.3 Modal Operators: Interior and Closure . . . 9
8.4 Constructive Existence: Tags as Witnesses . . . 9
8.5 Martin-Löf Type Theory and Dependent Structure . . . 10
8.6 Computation as Topological Traversal . . . 10

9 Summary **11**

10 Conclusion **11**

1 Primitives

1.1 Tags

A **tag** is the atomic unit of the tagd system. All tokens in TAGL are UTF-8 labels [7]. A tag carries two distinct identities:

- **Nominal identity**: a globally unique UTF-8 label.
- **Structural identity**: its position in the tagspace hierarchy, given by its rank, defined by its subordinate relation (aka identity relation) [5].

Definition 1.1 (Tag Set)**.** *Let T be the set of all tags in a tagspace. Each tag $t \in T$ is identified by a string* $\mathrm{id}(t) \in \Sigma^*$*, where Σ^* denotes the set of all finite UTF-8 strings.*

In TAGL, defining a tag looks like:

```
>> dog _is_a mammal;
```

Listing 1: Defining a tag via a subordinate relation

Here `dog` is a tag. Its nominal identity is the string `"dog"`. Its structural identity is defined by its subordinate relation to `mammal`.

1.2 The Root and the Axiom

Every tagspace has a distinguished root tag: `_entity`. The README states:

> `_entity` is axiomatic and self-referencing (i.e. `_entity _sub _entity`). It is the only tag having this relation. It is the root of a tagspace.

Definition 1.2 (Axiomatic Root)**.** *The tag `_entity` is the **axiomatic root** of every tagspace. It is the unique fixed point of the subordinate relation: the one tag whose super-object is itself.*

In TAGL notation:

```
_entity _sub _entity
```

Listing 2: The axiomatic root

In set-theoretic terms, `_entity` is the **least element** of the tagspace under the subordinate order — the tag that is an ancestor of everything, including itself.

2 The Subordinate Relation and the Rooted Tree

2.1 Subordinate Relations

A tag's structural identity is defined by exactly one subordinate (sub) relation. The TAGL specification states [5]:

> A subject MUST have exactly one subordinate relation to exist in the tagspace.

The sub relators are defined in `hard-tags.h` [1]:

```
#define HARD_TAG_SUB     "_sub"     // most abstract sub relator
#define HARD_TAG_IS_A    "_is_a"    // instance subordination
#define HARD_TAG_TYPE_OF "_type_of" // type subordination
```

Listing 3: Sub relator definitions in hard-tags.h

A typical TAGL knowledge hierarchy:

```
>> physical_object -^ _entity;
>> living_thing _type_of physical_object;
>> animal _type_of living_thing;
>> mammal _type_of animal;
>> dog _is_a mammal;
```

Listing 4: Building a subordinate hierarchy in TAGL

2.2 The Tagspace as a Rooted Tree

The subordinate relation organizes all tags into a **rooted tree**.

Definition 2.1 (Tagspace Tree)**.** *A tagspace is a pair* (T, p) *where:*

- T *is a set of tags containing the distinguished root* `_entity`*,*
- $p : T \setminus \{\texttt{_entity}\} \to T$ *is the* parent function*, assigning each non-root tag its super-object.*

There are no cycles (except the axiomatic self-reference of `_entity`*).*

The tree structure of the example above:

```
_entity
  physical_object
    living_thing
      animal
        mammal
          dog
```

This is not merely a data structure choice. The tree encodes *taxonomic knowledge*: to be a `dog` is to be a `mammal`, which is to be an `animal`, and so on. Structural position *is* meaning.

3 Rank: Encoding the Tree in Strings

3.1 The Rank Function

Each tag is assigned a **rank** — a UTF-8 byte string encoding its position in the tagspace tree. The `rank.h` source states:

> A rank string is the ordinal value of each node in the tagspace tree. A code point at a particular level is known as the rank value.

Definition 3.1 (Rank Function)**.** *The rank function $r : T \to \Sigma^*$ assigns to each tag a UTF-8 string such that the string's structure reflects the tag's position in the tree.*

Concrete rank assignments from the bootstrap dump [8]:

Tag	Rank (dotted)	Position
`_entity`	(empty)	level 0, root
`_sub`	1	level 1
`_is_a`	1.1	level 2
`_type_of`	1.2	level 2
`_rel`	2	level 1
`_has`	2.1	level 2
`_number`	3	level 1
`_integer`	3.1	level 2
`_float`	3.2	level 2
`_interrogator`	4	level 1

3.2 The Prefix Partial Order

The key property of ranks is stated explicitly in `rank.h`:

> All children of a particular node in the rank hierarchy will be prefixed by the parent node — that is, rank substrings define subtrees in the hierarchy.

Definition 3.2 (Containership Relation)**.** *Define the* ***containership relation*** $\sqsubseteq$ *on T by:*

$$a \sqsubseteq b \iff r(b) \text{ is a prefix of } r(a).$$

That is, $a \sqsubseteq b$ means b is an ancestor of a (or $b = a$). Tag b contains *tag a in the hierarchy.*

This relation is implemented directly in `rank.h`:

```
// this rank contains the rank being compared,
// in other words, this rank is a prefix to the other
bool contains(const rank&) const;
```

Listing 5: Containership via rank prefix in rank.h

And used in `tagd.cc` for structural operations on tag sets:

```
if (b->rank().contains(a->rank())) { // rank can contain itself (b == a)
    merged++;
} else if (a->rank().contains(b->rank())) {
    A.insert(*b);
}
```

Listing 6: Containership used in merge_containing_tags (tagd.cc)

3.3 Verifying the Partial Order Axioms

Theorem 3.1. *The containership relation $\sqsubseteq$ is a partial order on T.*

Proof. We verify the three required properties.

Reflexivity. `rank.h` notes that `contains()` holds when $b = a$, since every string is a prefix of itself. Hence $a \sqsubseteq a$ for all $a \in T$.

Antisymmetry. Suppose $a \sqsubseteq b$ and $b \sqsubseteq a$. Then $r(b)$ is a prefix of $r(a)$ and $r(a)$ is a prefix of $r(b)$. A string u is a prefix of v and v is a prefix of u if and only if $u = v$. Hence $r(a) = r(b)$, which implies $a = b$ since ranks uniquely identify tags.

Transitivity. Suppose $a \sqsubseteq b$ and $b \sqsubseteq c$. Then $r(b)$ is a prefix of $r(a)$ and $r(c)$ is a prefix of $r(b)$. Since the prefix relation on strings is transitive, $r(c)$ is a prefix of $r(a)$, so $a \sqsubseteq c$. □

Therefore $(T, \sqsubseteq)$ is a **poset** (partially ordered set).

3.4 Lexicographic Total Order

Ranks also carry a total **lexicographic order**, implemented in `rank.h`:

```
bool operator<(const rank& rhs) const { return _data < rhs._data; }
```

Listing 7: Lexicographic ordering of ranks

This order — a deliberate UTF-8 design choice noted in `rank.h` ("it collates lexicographically") — enables efficient storage and retrieval of the entire tagspace as a sorted structure. Tags within the same subtree sort together. The tree *is* the index.

4 The Tagspace Topology

4.1 From Partial Order to Topology

Every partial order canonically generates a topology. Given the poset $(T, \sqsubseteq)$, we define:

Definition 4.1 (Tagspace Open Sets)**.** *A subset $U \subseteq T$ is **open** if it is upward-closed under $\sqsubseteq$:*

$$U \in \mathcal{T} \iff \forall\, a \in U,\ \forall\, b \in T:\ a \sqsubseteq b \Rightarrow b \in U.$$

In the tagspace tree, "upward-closed" means: if a tag is in the set, all its ancestors are too. Each basic open set is the **subtree rooted at a tag** together with all tags above it.

This construction is known as the **Alexandrov topology** — the topology generated by a partial order in which arbitrary intersections (not just finite ones) of open sets remain open.

4.2 Verifying the Topology Axioms

Theorem 4.1. *Let M be the set of all tags in a tagspace and let $\mathcal{T}$ be the collection of upward-closed subsets under $\sqsubseteq$. Then $(M, \mathcal{T})$ is a topological space.*

Proof. We verify the three axioms.

(i) $\emptyset \in \mathcal{T}$ and $M \in \mathcal{T}$. The empty set is vacuously upward-closed. The full set M is upward-closed because there is no $b \in T$ outside M. Both belong to $\mathcal{T}$.

(ii) Finite intersections. Let $U, V \in \mathcal{T}$. Suppose $a \in U \cap V$ and $a \sqsubseteq b$. Then $b \in U$ (since U is upward-closed) and $b \in V$ (since V is upward-closed), so $b \in U \cap V$. Hence $U \cap V \in \mathcal{T}$.

(iii) Arbitrary unions. Let $\{U_\alpha\}$ be any collection of sets in $\mathcal{T}$. Suppose $a \in \bigcup_\alpha U_\alpha$ and $a \sqsubseteq b$. Then $a \in U_\alpha$ for some α. Since U_α is upward-closed, $b \in U_\alpha \subseteq \bigcup_\alpha U_\alpha$. Hence $\bigcup_\alpha U_\alpha \in \mathcal{T}$. □

Therefore $(M, \mathcal{T})$ is a **topological space**. The `merge_containing_tags` function in `tagd.cc` computes exactly the intersection of two subtrees, exhibiting axiom (ii) in code:

```
if (b->rank().contains(a->rank())) {
    merged++;
} else if (a->rank().contains(b->rank())) {
    A.insert(*b);
}
```

Listing 8: Intersection of subtrees in tagd.cc

4.3 Specialization Order

In the Alexandrov topology, every topological space carries a natural **specialization preorder**: a specializes b (written $a \rightsquigarrow b$) when a is in the topological closure of $\{b\}$.

Proposition 4.1. *In the tagspace topology, the specialization order coincides with the subordinate order: $a \rightsquigarrow b$ if and only if a is a descendant of b in the tagspace tree.*

In plain language: `dog` specializes `mammal`, which specializes `animal`. More specific tags (deeper in the tree) specialize more general ones. Taxonomic inheritance is not a rule to be enforced — it is a geometric consequence of the shape of the space.

4.4 Continuity as Meaning-Preservation

Definition 4.2 (Tagspace Map)**.** *A map $f : T_1 \to T_2$ between two tagspaces is* ***topologically continuous*** *if it preserves open sets: $f^{-1}(U) \in \mathcal{T}_1$ for every $U \in \mathcal{T}_2$.*

Proposition 4.2. *A map $f : T_1 \to T_2$ is continuous (with respect to the Alexandrov topologies) if and only if it is order-preserving: whenever $a \sqsubseteq b$ in T_1, we have $f(a) \sqsubseteq f(b)$ in T_2.*

This gives a precise mathematical definition of a **meaning-preserving transformation** between tagspaces. If `dog _is_a mammal` in T_1, a continuous map must send `dog` somewhere below the image of `mammal` in T_2.

5 The Hard Tag Base: A Universal Subspace

5.1 Hard Tags

Every tagspace — no matter how different its user-defined tags — contains exactly the same set of hard tags, defined in `hard-tags.h` [1]:

```
#define HARD_TAG_ENTITY       "_entity"       // root
#define HARD_TAG_SUB          "_sub"          // subordination
#define HARD_TAG_IS_A         "_is_a"         // instance relation
#define HARD_TAG_TYPE_OF      "_type_of"      // type relation
#define HARD_TAG_RELATOR      "_rel"          // predicate root
#define HARD_TAG_HAS          "_has"          // possession
#define HARD_TAG_CAN          "_can"          // capability
#define HARD_TAG_NUMBER       "_number"       // numeric type root
#define HARD_TAG_INTEGER      "_integer"      // integer type
#define HARD_TAG_FLOAT        "_float"        // float type
#define HARD_TAG_INTERROGATOR "_interrogator" // query root
```

Listing 9: Core hard tag definitions in hard-tags.h

5.2 The Hard Tag Subspace

Definition 5.1 (Hard Tag Subspace)**.** *Let $H \subset T$ denote the set of hard tags in any tagspace T. The induced subspace topology on H is identical in every tagspace. H is therefore the* ***universal invariant subspace*** *— a fixed topological base that every tagspace shares by construction.*

Remark 5.1. *In categorical terms, the hard tag subspace is the* ***initial object*** *in the category of tagspaces: every tagspace is an extension of H, and every valid map between tagspaces must fix H.*

5.3 Interoperability via the Common Base

Because every tagspace contains H as a common topological foundation, any two tagspaces can communicate. The hard tags are the shared language — the invariant ground that makes the `httagd` web protocol possible. Two tagspaces may diverge arbitrarily in their user-defined tags, but they always agree on the structure of H.

```
-- invariant in every tagspace:
_entity _sub _entity

-- user-defined, varies by tagspace:
>> dog _is_a mammal;
```

Listing 10: Hard tags are invariant; user tags vary

6 Predicates: Fibered Structure

6.1 Relators Are Tags

Crucially, relators are themselves tags — subordinate to `_rel` in the tagspace tree. This means predicates are not outside the system; they are *inside* it. The tagspace is self-describing.

From the bootstrap dump [8]:

```
_rel     rank: 2        (root of all relators)
_has     rank: 2.1
_can     rank: 2.2
_refers  rank: 2.3
```

The language used to attach meaning to tags is itself expressed in the same structural language.

6.2 Predicates as Horizontal Relations

Predicates are relations attached to a tag that do not alter its position in the tree. From `TAGL-spec.md` [5]:

> A predicate consists of a relator followed by an object_list. A relator MUST be a tag subordinate to `_rel`.

```
>> dog _is_a mammal     -- vertical: places dog in the tree
    _has legs = 4, tail
    _can bark;          -- horizontal: predicate data at dog
```

Listing 11: Vertical identity and horizontal predicates

6.3 The Two-Layer Structure

The tagspace carries two distinct mathematical layers:

Definition 6.1 (Two-Layer Tagspace Structure). *1. **Base space**: the topological space $(M, \mathcal{T})$ defined by subordinate relations and ranks — the vertical dimension.*

*2. **Fiber**: at each tag $t \in M$, a set of predicates $P(t) \subseteq \mathrm{Rel} \times T \times \mathrm{Mod}$ — the horizontal dimension, where* Rel *is the set of relator tags and* Mod *is the set of modifier values.*

This two-layer structure — a base space with additional data attached at each point — is analogous to a **fiber bundle** in differential geometry. The topology gives the skeleton; the predicates give the flesh.

The `abstract_tag` class in `tagd.h` encodes exactly this decomposition:

```
class abstract_tag {
    tagd::rank    _rank;        // position in the base space
    predicate_set relations;    // fiber: additional data at this point
};
```

Listing 12: Two-layer structure in abstract_tag (tagd.h)

7 Structural Identity

7.1 Identity Is Position

The deepest mathematical claim of the tagd system is that a tag's identity is entirely structural. The TAGL specification defines:

> **Structural identity**: its position in the tagspace hierarchy, given by its rank, defined by its subordinate relation (aka identity relation).

A tag `dog` is not `dog` because of the string `"dog"`. It is `dog` because of where it sits — under `mammal`, under `animal`, under `living_thing`, under `_entity` — and because of what predicates are attached to it there. Change the position (the subordinate relation), and you have a different tag, regardless of the label.

This is **mathematical structuralism** applied to knowledge representation: identity is relational, not intrinsic.

7.2 The Canonical Form

A tagspace can be fully serialized as TAGL text via `dump()`. From the TAGL Thesis document:

> The canonical form of a tagspace is its dump — the UTF-8 TAGL code that represents the tagspace (and can be loaded).

The dump is not a record of a structure. It *is* the structure, expressed in TAGL. The tagspace is its own specification.

8 Deeper Structures

The foundations established in Sections 1–8 are sufficient to describe what the tagd system *is*. This section describes what it *implies* — deeper mathematical structures that emerge from those foundations and illuminate how the system behaves.

8.1 The Adjunction Between Order and Topology

The relationship between the tagspace as a partial order and the tagspace as a topological space is not incidental. It is a formal **adjunction** — one of the most fundamental relationships in category theory.

Two functors mediate between the two views:

- $A : \mathbf{PreOrd} \to \mathbf{Top}$ (the *Alexandrov functor*) takes the rank prefix partial order on tags and produces the Alexandrov topological space described in Section 4.
- $S : \mathbf{Top} \to \mathbf{PreOrd}$ (the *specialization functor*) takes any topological space and recovers a preorder: $a \leq b$ iff every open set containing a also contains b.

These functors form an adjunction: A is left adjoint to S, written $A \dashv S$. Applying both in sequence recovers the original structure: $S(A(P)) \cong P$.

In tagd, this adjunction is visible in the two-step construction of a fully hydrated tag. A tag begins as a semantic object — identity, subordinate relation, position — living in the preorder. When its rank is assigned, it is lifted into the Alexandrov topological space. The rank upgrade is the **unit of the adjunction**: the reflection morphism that moves a tag from the order-theoretic world into the topological world while preserving its identity on the underlying set.

```
// Semantic tag: lives in PreOrd
tagd::abstract_tag semantic(id, sub_relator, super_object, pos);

// Reflection map: lifts into the Alexandrov space
tagd::abstract_tag hydrated = rank.empty()
    ? std::move(semantic)
    : tagd::abstract_tag(semantic, rank);  // <- unit of the adjunction
```

Listing 13: The reflection morphism: lifting a tag from PreOrd into the Alexandrov space

The reflection morphism is:

- **Bijective** on the underlying set of tags (same identity, same element)
- **Continuous** with respect to the Alexandrov topology
- **Order-reflecting** (preserves and reflects the subordinate relation via rank prefixes)

This is a **bimorphism** in the category of preorders and topological spaces.

8.2 Topological Properties of the Tagspace

Because the tagspace carries an Alexandrov topology, it inherits a family of topological properties that have direct operational consequences.

First countable. Every tag x has a smallest open neighbourhood: the set of all tags whose rank has $r(x)$ as a prefix — all descendants of x plus x itself. This smallest neighbourhood is the local base at x. It is why rank-based traversal (`get_children`, `related`) is efficient: the topology guarantees a finite, well-defined local structure at every point.

Locally compact. For any tag x, the smallest open neighbourhood $U(x)$ is compact in the Alexandrov topology. Any open cover of $U(x)$ must contain $U(x)$ itself, giving a trivial finite subcover. This is why queries over subtrees are well-behaved: every neighbourhood is compact by construction.

Locally path connected. Between any two comparable tags $x \sqsubseteq y$, there is a path up the subordinate chain via successive super-object steps. The smallest neighbourhood of any tag is connected (it is an upper set), so the space is locally path-connected in the order-theoretic sense.

Subspaces and products are Alexandrov-discrete. Any subset of tags — for example, the result of a `query()` or all descendants of a given tag — inherits the Alexandrov structure. Query results compose correctly because they are subspaces of the same topological space, not alien data structures.

8.3 Modal Operators: Interior and Closure

In any topological space, the **interior** and **closure** operators are well-defined. In the tagspace, they carry semantic meaning.

Topological operator	Modal logic	tagd meaning
Interior $\mathrm{int}(U)$	Necessarily ($\Box$)	Properties true in all superiors of a tag
Closure $\mathrm{cl}(U)$	Possibly ($\Diamond$)	Properties reachable via any subordinate

The tagspace is therefore an **interior algebra** — a Boolean algebra equipped with an interior operator satisfying the Kuratowski axioms. The `related()`, `has_relator()`, and `get_children()` families of functions are the operational realization of these modal operators.

Furthermore, because the Alexandrov topology arises from a partial order, the tagspace constitutes a **Kripke frame**: a set of worlds (tags) equipped with an accessibility relation (the subordinate order). A predicate P is *necessarily true* at tag x (written $\Box P(x)$) if P holds at every tag above x. It is *possibly true* ($\Diamond P(x)$) if P holds at some tag below x.

```
-- "dog necessarily _has legs" if all ancestors of dog also _has legs
-- "animal possibly _can bark" if some descendant of animal _can bark
```

Listing 14: Modal reading of TAGL predicates

This gives TAGL queries a natural modal interpretation without any additional machinery. The topology provides the modal semantics for free.

8.4 Constructive Existence: Tags as Witnesses

The trail from Kripke semantics through intuitionistic logic leads to a foundational observation about how tagd represents knowledge.

In **classical mathematics**, an existence proof may assert "there exists an object with property P" without producing one. In **constructive mathematics**, existence requires a witness — an explicit object that demonstrates the property.

The tagd system is constructive by design. A tag does not *exist* in the tagspace until it is explicitly constructed with a subordinate relation. The TAGL rule — *a tag MUST have exactly one subordinate relation* — is a constructive existence condition. No witness, no tag.

After the const-correctness work (Phase 9B), this constructive character is enforced in the type system: tags are built once, immutably, in a single construction step. There is no "assert existence, hydrate later" pattern. The construction *is* the proof of existence.

```
// Constructive: the tag is its own existence proof.
// No tag exists until this line executes.
tagd::abstract_tag t(id, sub_relator, super_object, pos);
```

Listing 15: One-shot construction as constructive existence

This aligns the implementation with **intuitionistic logic**, where a proof of $\exists x.\, P(x)$ is a pair (t, p) — a term t and a proof p that $P(t)$ holds. In tagd: the term is the tag, and the proof is its rank and subordinate relation.

8.5 Martin-Löf Type Theory and Dependent Structure

The constructive character of tagd points toward **Martin-Löf Type Theory** (MLTT) — the most rigorous modern foundation for constructive mathematics, and the theoretical basis for proof assistants such as Agda, Lean, and Coq.

In MLTT, propositions are types and proofs are programs (the Curry-Howard correspondence). Every existence claim must be witnessed by an explicit term inhabiting the corresponding type.

The tagspace can be read as a fragment of MLTT:

MLTT concept	**tagd realization**
Type	Tag position in the tagspace (defined by rank)
Term inhabiting a type	A concrete tag at that position
Dependent type	Rank-dependent tag structure
Sigma type $\Sigma_{x:A}\, B(x)$	A tag together with its predicate fiber
Identity type	Immutable structural identity: same rank = same type
Universe	The tagspace as a whole
Constructive proof	One-shot tag construction; `dump()` as canonical form

The rank function $r : T \to \Sigma^*$ is a **dependent type family**: the structural type of a tag depends on its rank value. Two tags at different ranks are, in the type-theoretic sense, of different types — even if their nominal labels are similar.

Remark 8.1. *Making identity immutable in the C++ type system (Phase 9B) is not merely a software quality improvement. It is a* ***formalization step****: the C++ types now enforce what MLTT would express as an identity type. Once a tag's rank cannot be mutated, the fixed-point property of* `_entity` *is not just documented — it is an axiom enforced by the compiler.*

8.6 Computation as Topological Traversal

The introduction of functions as tags in TAGL (Section 7 of the TAGL specification) completes the arc from the tagspace as a knowledge structure to the tagspace as a computing environment. The mathematical consequences are significant.

Functions are tags. Defining a function is a `>>` PUT operation — a constructive existence claim. The function does not exist until it is placed in the tagspace with a subordinate relation. It is subject to the same identity rules, the same rank structure, and the same topological properties as any other tag. It can be queried, inspected, versioned, and compared.

```
>> greet($name) _is_a function
    -> "Hello, " ++ $name;
```

Listing 16: A function defined as a tag via PUT

This is not syntactic convenience. It means the computation is *inside* the topological space, not outside it.

A function call is a topological operation. A call navigates from the function tag (a point in T) through its predicate fiber (the body) to a result. Combined with the Curry-Howard correspondence from Section 8.5: *tags are types, functions are proofs, and evaluation is topological traversal.*

Single-assignment variables are proof terms. Variables (`$name`) follow single-assignment semantics — immutable after binding. This is directly consistent with the constructive / MLTT foundation of Section 8.4: a variable binding is a term inhabiting a type. Once constructed, it cannot be mutated, for the same reason a tag's rank cannot be mutated after construction.

Guards are modal assertions. The `_when` guard syntax exposes the Kripke frame structure of the tagspace directly in the language. A guard using $\Box$ asserts necessity:

```
>> area($shape) _when [](has radius) ->
    3.14159 * $shape.radius ** 2;
```

Listing 17: Modal guard using the necessity operator

This asserts that the function applies only when `_has radius` is *necessarily* true — true at this tag and all its ancestors in the Alexandrov topology. The modal operators $\Box$ and $\Diamond$ are not new machinery; they are the interior and closure operators of Section 8.3, exposed as first-class language constructs.

Async processes communicate via continuous maps. The Erlang-style process model in TAGL sends TAGL text through buffers between isolated processes. Topologically, a channel between two tagspace-aware processes is a *continuous map* — an order-preserving transformation that preserves the subordinate structure. The hard tag base H is the common topological ground that makes inter-process TAGL intelligible: both ends of a channel share the universal invariant subspace, guaranteeing that the structural meaning of hard-tag constructs is preserved across the channel boundary.

The three-part alignment is now complete:

Layer	**Structure**	**Mathematical object**
Knowledge	Tags and subordinate relations	Alexandrov topological space
Meaning	Predicates at each tag	Fiber over the base space
Computation	Functions as tags, evaluation as traversal	Topological map + Kripke semantics

9 Summary

Table 1 collects all mathematical structures identified in the tagd source, including those from the deeper structures of Section 9.

10 Conclusion

The tagd system is not merely a database or a knowledge graph. It is a **topological space of structured meaning**. Tags are points. Subordinate relations give the space its shape. Ranks encode that shape as a partial order. The Alexandrov topology makes the shape mathematically precise. Predicates attach additional structure at each point as a fiber over the base topology. And the hard tag base ensures that every tagspace shares the same foundational geometry.

TAGL is the language for acting on this space: defining points, querying regions, transforming structure. Every TAGL statement is a topological operation.

When functions enter the tagspace as tags, computation becomes part of the same structure. A function call is a traversal from a point in T through its predicate fiber to a result. Guards express modal assertions over the Kripke frame. Async processes communicate via continuous maps across the shared hard tag base. Knowledge, meaning, and computation are no longer separate concerns — they are three layers of the same topological object.

The Curry-Howard correspondence completes the picture: tags are types, functions are proofs, and evaluation is topological traversal.

The vision that *information is structural* — that meaning comes from shape, not from labels — is not a design philosophy. It is a theorem about how the system works.

Mathematical Structure	tagd Realization	Source
Set	All tags T in a tagspace	`tagd.h: tag_set`
Rooted tree	Subordinate relations from `_entity`	`tagd-README.md`
Partial order (poset)	Rank prefix containership $\sqsubseteq$	`rank.h: contains()`
Lexicographic total order	Rank byte string comparison	`rank.h: operator<`
Alexandrov topology	Upward-closed subtrees as open sets	`rank.h`, `tagd.cc`
Universal base subspace	Hard tag set H	`hard-tags.h`
Fiber / attached structure	Predicate sets $P(t)$ at each tag	`tagd.h: predicate_set`
Continuous map	Order-preserving tagspace map	`tagd.cc: merge_containing_tags`
Canonical serialization	`dump()` in TAGL canonical form	`TAGL-spec.md`
Adjunction ($A \dashv S$)	Rank upgrade as reflection morphism	`tagd.cc: abstract_tag` ctor
First countable	Smallest open nbhd via rank prefix	`rank.h: contains()`
Locally compact	Subtree queries always well-behaved	`tagd.cc: get_children`
Interior algebra / modal ops	`related()`, `has_relator()` as $\Box/\Diamond$	`tagd.h`
Kripke frame	Subordinate order as accessibility relation	`rank.h`, `tagd.cc`
Constructive existence	One-shot tag construction as witness	`tagd.h: abstract_tag` ctor
Dependent type (MLTT)	Rank-dependent tag structure	`rank.h`, `tagd.h`
Functions as tags	PUT-defined executable tags in tagspace	`TAGL-spec.md` §7
Computation as traversal	Function call as topological map through fiber	`TAGL-spec.md` §7
Modal guards	`_when` $\Box/\Diamond$ as interior/closure in syntax	`TAGL-spec.md` §7.4

Table 1: Mathematical structures and their tagd source realizations.

References

[1] tagd source code, `tagd/include/tagd/hard-tags.h`. Hard tag definitions and axioms.

[2] tagd source code, `tagd/include/tagd/rank.h`. Rank structure, prefix order, and `contains()` implementation.

[3] tagd source code, `tagd/include/tagd.h`. `abstract_tag`, `predicate_set`, and tag set operations.

[4] tagd source code, `tagd/src/tagd.cc`. `merge_containing_tags` and merge operations.

[5] *TAGL Specification*, `TAGL-spec.md`. Language specification and semantic model.

[6] *TAGL Design Thesis*, `TAGL-Thesis.md`. Language design document including function model and computation semantics.

[7] *tagd README*, `tagd-README.md`. Working examples and TAGL tutorial.

[8] *Bootstrap Dump Grid*, `bootstrap-dump-grid.txt`. Concrete tagspace rank assignments.